ASSIGNMENT 2 – CO2

Compiler Design: Parsing and Syntax Analysis

Total Marks: 50 | Individual Assignment

Student Name: N. Pujitha Naga Sai Lakshmi

Register Number: 192572119

Q1 – Designing a Grammar that Means One Thing (6 Marks)

1.1 Role of the parser and regular-expression limitation

A lexical analyzer converts the character stream into tokens such as IF, ID, NUMBER, AND, NOT, relational operators, WATER, and semicolon. A parser receives this token stream and checks whether it follows the syntax of the rule language. It builds a parse tree or AST, reports syntax errors, and provides structured information for semantic analysis and symbol-table services. A regular expression is not suitable for the entire language because nested parentheses and recursively nested statements require unbounded matching; this is a context-free property.

1.2 Grammar G = (V, T, P, S)

```
V = {Program, Stmt, ElsePart, Cond, Or, And, Not, Rel, Expr, Term, Factor}
T = {if, else, water, (, ), ;, &&, ||, !, <, >, <=, >=, ==, !=, +, -, *, /, id, num}
S = Program

Program  -> Stmt
Stmt     -> if ( Cond ) Stmt ElsePart
          | water Expr ;
ElsePart -> else Stmt | ε
Cond     -> Or
Or       -> Or || And | And
And      -> And && Not | Not
Not      -> ! Not | Rel
Rel      -> Expr RelOp Expr | ( Cond )
RelOp    -> < | > | <= | >= | == | !=
Expr     -> Expr + Term | Expr - Term | Term
Term     -> Term * Factor | Term / Factor | Factor
Factor   -> ( Expr ) | id | num
```

For predictive/top-down use, eliminate left recursion in Or, And, Expr, and Term. Precedence is encoded by the levels: ! binds most tightly, then relational operations, then &&, then ||. Arithmetic has multiplication/division above addition/subtraction.

1.3 Valid rule and derivation

Chosen rule: water 20;

```
Leftmost derivation:
Program => Stmt => water Expr ; => water Term ;
      => water Factor ; => water num ; => water 20 ;

Rightmost derivation (same chain because only one nonterminal is present):
Program => Stmt => water Expr ; => water Term ;
      => water Factor ; => water num ; => water 20 ;
```

Parse tree:

```
     Program
       |
      Stmt
    /  |   \
 water Expr  ;
       |
      Term
       |
     Factor
       |
      num(20)
```

1.4 Ambiguity and repair

A deliberately ambiguous expression grammar is E -> E + E | E * E | id. For id + id * id, two structures are possible: (id + id) * id and id + (id * id). The grammar is repaired by separating precedence levels: E -> E + T | T; T -> T * F | F; F -> id. This preserves the intended set of arithmetic expressions while enforcing multiplication precedence.

```
Tree A:              Tree B:
     *                    +
    / \                  / \
   + id                id  *
  / \                     / \
 id id                   id id
```

The dangling-else ambiguity occurs when an else can attach to either a nearer or farther if. It can be removed by the matched/unmatched statement grammar, or by explicit braces/block delimiters. For this safety-critical controller, ambiguity can cause a different valve action; therefore a deterministic grammar and AST are required. A proof of unambiguity requires a formal argument; testing several strings only provides evidence, not a proof.

1.5 Representation decision

Choose an abstract syntax tree (AST). It removes punctuation and grammar-only nodes while preserving operators, conditions, and action statements. Semantic analysis can validate variable names and moisture ranges, and code generation can map the AST to controller operations. Evaluating during parsing is rejected because it couples syntax recognition to hardware actions and complicates safety checks.

Q2 – Top-Down Parsing: From Grammar to Working Parser (8 Marks)

2.1 Why naïve recursive descent fails

```
E() { E(); match('+'); T(); }  // follows E -> E + T first
```

Calling E immediately calls E again without consuming input. This is direct left recursion and causes unbounded recursion/stack overflow. The standard transformation removes left recursion while preserving the language.

```
Original: E -> E + T | E - T | T
          T -> T * F | T / F | F
          F -> (E) | id | num

Transformed:
E  -> T E'
E' -> + T E' | - T E' | ε
T  -> F T'
T' -> * F T' | / F T' | ε
F  -> ( E ) | id | num
```

The transformed grammar generates the same arithmetic strings. The AST builder folds E-prime operations from left to right, preserving left associativity: a-b-c becomes (a-b)-c.

2.2 Left factoring and FIRST/FOLLOW

```
S -> if C then S S' | id = E
S' -> else S | ε
```

This factoring exposes the common prefix if C then S. A conventional matched/unmatched grammar is preferred when the language must specify nearest-if binding exactly.

Nonterminal	FIRST	FOLLOW (representative)
E	{ (, id, num }	{), $, ; }
E'	{ +, -, ε }	{), $, ; }
T	{ (, id, num }	{ +, -,), $, ; }
T'	{ *, /, ε }	{ +, -,), $, ; }
F	{ (, id, num }	{ *, /, +, -,), $, ; }

FIRST is obtained by adding terminals that can begin each production; FOLLOW is seeded with $ for the start symbol and propagated through each production until no set changes. The LL(1) table places A -> α in M[A,a] for a in FIRST(α), and for ε-productions also uses FOLLOW(A).

2.3 Recursive-descent code and actual run

The runnable Python prototype included with this document parses the expression rate * (p + f) - d, constructs an AST, and rejects an expression with a missing operand. It implements the transformed E/T/F precedence structure.

```
import re

TOKENS = re.compile(r"\s*(?:(\d+)|([A-Za-z_][A-Za-z_0-9]*)|(\*|/|\+|-|\(|\)))")
class ParseError(Exception): pass

class Parser:
    def __init__(self, text):
        self.text=text; self.tokens=[]; pos=0
        while pos < len(text):
            m=TOKENS.match(text,pos)
            if not m: raise ParseError(f"invalid character at position {pos}: {text[pos]!r}")
            self.tokens.append(m.group(1) or m.group(2) or m.group(3)); pos=m.end()
        self.tokens.append('$'); self.i=0
    def peek(self): return self.tokens[self.i]
    def take(self, expected=None):
        t=self.peek()
        if expected is not None and t != expected: raise ParseError(f"expected {expected},
found {t}")
```

```
        self.i += 1; return t
    def E(self):
        node=self.T()
        while self.peek() in ('+','-'):
            op=self.take(); right=self.T(); node=(op,node,right)
        return node
    def T(self):
        node=self.F()
        while self.peek() in ('*','/'):
            op=self.take(); right=self.F(); node=(op,node,right)
        return node
    def F(self):
        if self.peek()=='(': self.take('('); n=self.E(); self.take(')'); return n
        t=self.peek()
        if re.fullmatch(r'\d+',t): self.take(); return int(t)
        if re.fullmatch(r'[A-Za-z_][A-Za-z_0-9]*',t): self.take(); return ('id',t)
        raise ParseError(f"expected number, identifier, or (, found {t}")
    def parse(self):
        n=self.E()
        if self.peek()!='$': raise ParseError(f"unexpected token {self.peek()}")
        return n

def show(n):
    if isinstance(n,int): return str(n)
    if n[0]=='id': return n[1]
    return f"({show(n[1])} {n[0]} {show(n[2])})"

def check_rule(s):
    # Tiny scanner-level validation of sample irrigation statements.

pattern=r"^if\s*\(\s*moisture\s*(<|>)\s*\d+\s*&&\s*!\s*raining\s*\)\s*water\s+\d+\s*;\s*
$"
    return bool(re.match(pattern,s))

if __name__=='__main__':
    expr='rate * (p + f) - d'
    tree=Parser(expr).parse()
```

```
print('Recursive-descent parser')
print('Input :',expr)
print('AST   :',show(tree))
print('Result: ACCEPT')
print('\nError test')
bad='rate * (p + ) - d'
try: Parser(bad).parse()
except ParseError as e: print('Input :',bad); print('Result: REJECT -',e)
print('\nIrrigation-rule lexical check')
rule='if (moisture < 30 && !raining) water 20;'
print('Input :',rule)
print('Result:', 'ACCEPT' if check_rule(rule) else 'REJECT')
```

Call sequence for rate * (p + f) - d: E → T → F(rate), T consumes *, F consumes parenthesized E(p+f), then E consumes - and F(d).

2.4 Predictive stack trace (compact)

Stack (top at right)	Input	Action
$ E	rate * (p + f) - d $	E → T E-prime
$ E-prime T	rate * (p + f) - d $	T → F T-prime
...	...	match rate; then match *
...	(p + f) - d $	F → (E)
...	p + f) - d $	parse inner E
$	$	ACCEPT

A missing operand such as rate * (p +) - d is detected when F expects a number, identifier, or opening parenthesis but sees). The parser reports the unexpected token at that point.

2.5 Backtracking, LL(1), and design decision

Backtracking may repeatedly parse the same prefix. For nested alternatives, the number of explored branches can grow exponentially in nesting depth in the worst case. LL(1) requires disjoint FIRST sets for alternatives and, for ε, disjointness from FOLLOW. Example non-LL(1): A -> a b | a c; both alternatives begin with a, so M[A,a] has two entries. Factoring reduces this conflict. For future function calls name(args), top-down parsing remains convenient if the grammar is factored and function-call parsing is handled as a suffix in the primary-expression rule.

Q3 – Bottom-Up Parsing: Finding What to Reduce (6 Marks)

3.1 Shift-reduce sequence and handles

Grammar: E -> E+T | T; T -> T*F | F; F -> (E) | id. Input: id + id * id. A handle is the substring matching the RHS of a production whose reduction is one step in the reverse of a rightmost derivation.

Stack	Input	Action
$	id + id * id $	shift id
$ id	+ id * id $	reduce F → id
$ F	+ id * id $	reduce T → F
$ T	+ id * id $	reduce E → T
$ E	+ id * id $	shift +
$ E +	id * id $	shift id
$ E + id	* id $	reduce F → id
$ E + F	* id $	reduce T → F
['$ E + T']	* id $	shift *
['$ E + T *']	id $	shift id
['$ E + T * id']	$	reduce F → id
['$ E + T * F']	$	reduce T → T * F
['$ E + T']	$	reduce E → E + T
['$ E']	$	ACCEPT

3.2 Conflicts

Shift/reduce conflict example: E -> E+E | E*E | id, with stack E+E and lookahead *. The parser can shift * or reduce E+E. Reduce/reduce example: S -> A | B; A -> id; B -> id. After id, both A -> id and B -> id may be reductions. Such grammars need structural redesign or precedence declarations.

3.3 Operator precedence

Use conventional precedence: ^ highest and right-associative; unary minus below exponentiation but above multiplication; then * and /; then + and -. Parentheses override precedence. In a precedence parser, for left-associative operators, the incoming operator yields precedence to the stack operator at equal level (reduce); for right-associative ^, equal precedence shifts. A compact relation summary is shown below; blank entries indicate delimiter/context cases handled by the parser boundary rules.

Stack terminal vs input	id	+	*	^	(	)	$
id	—	>	>	>	—	>	>
+	<	>	<	<	<	>	>
*	<	>	>	<	<	>	>
^	<	>	>	<	<	>	>
(	<	<	<	<	<	=	—
)	—	>	>	>	—	>	>
$	<	<	<	<	<	—	=

Operator-precedence parsing is unsuitable for grammars with adjacent nonterminals in RHS or ε-productions under the classical restrictions. Unary minus requires distinguishing prefix and infix forms, often with a dedicated UMINUS precedence. For firmware, a compact precedence parser can be efficient for arithmetic, but not as the sole parser once assignments, conditionals, and calls are added.

Q4 – LR Parsing and SLR Table (7 Marks)

4.1 Items, closure, and goto

Augment the grammar with S' -> S so the accept condition is explicit. An LR(0) item is a production with a dot showing how much of its RHS has been recognized. CLOSURE adds items for productions of the nonterminal immediately after the dot. GOTO(I,X) advances the dot over X in all items that allow it, then takes closure.

```
Augmented grammar:
0. S' -> S
1. S  -> C C
2. C  -> c C
3. C  -> d

I0 = closure({S' -> .S})
  S' -> .S
  S  -> .C C
  C  -> .c C
  C  -> .d

I1: S' -> S.
I2: S -> C . C; C -> .c C; C -> .d
I3: C -> c . C; C -> .c C; C -> .d
```

I4: C -> d.
I5: S -> C C.
I6: C -> c C.
DFA edges: I0 --S→ I1; I0 --C→ I2; I0 --c→ I3; I0 --d→ I4; I2 --C→ I5; I2 --c→ I3; I2 --d→ I4; I3 --C→ I6; I3 --c→ I3; I3 --d→ I4.

4.2 SLR table

State	ACTION c	ACTION d	ACTION $	GOTO S	GOTO C
0	s3	s4		1	2
1			acc		
2	s3	s4			5
3	s3	s4			6
4	r3	r3	r3		
5			r1		
6	r2	r2	r2		

FOLLOW(C) = {c, d, $} for this grammar, so completed C -> d and C -> cC reductions are entered only on FOLLOW(C). Valid input c d d $ is accepted; invalid input c c $ eventually reaches a state with no action for $, producing a syntax error.

4.3 Non-SLR grammar

S -> L = R | R
L -> * R | id
R -> L

The LR(0) state containing S -> L . = R and R -> L . can request shift on = and reduction R -> L. FOLLOW(R) includes =, so SLR may place both actions in the same cell. FOLLOW is global and too coarse; the specific LR(1) item lookahead can distinguish the contexts. LR(0) has least power and smallest tables; SLR adds FOLLOW filtering; LALR merges compatible LR(1) states; canonical LR(1) has the greatest power but often larger tables. Deterministic LR parsing is O(n) for fixed grammar. For 300 productions, use a parser-generator rather than hand-building states.

Q5 – Error Handling and Recovery (5 Marks)

5.1 Beginner error catalogue

Error input	LL(1) detection point	LR detection point
x = 5	after expression, expected ;	state has no action on next token
if (x) y=1; else	when else body is missing	after shifting else, no valid continuation

['x = (2+3;']	when) is expected	state rejects ;
['x = 2 + ;']	F sees ; unexpectedly	no action on ;
['else x=2;']	S cannot start with else	initial state has no action
['x == 3;']	assignment parser expects = or valid statement	state rejects comparison in assignment context

5.2 Recovery strategies

Panic-mode recovery skips tokens until a synchronizing token is found, typically FOLLOW(statement) = {; , }, else, $}. Phrase-level recovery inserts or deletes a small number of tokens near the error. Error productions explicitly accept common mistakes, such as statement -> error ;, and issue a tailored message. Every recovery step must either consume input, pop a stack symbol, or terminate; otherwise insertion can loop forever.

Method	Strength	Risk
Panic mode	Simple, progress is easy to enforce	May skip valid later code
Phrase-level	Local repair and clearer diagnostics	May guess wrong repair
['Error productions']	Helpful for known common mistakes	Grammar grows; may mask errors

For a beginner platform limited to three messages, use panic mode with statement-level synchronizers, suppress duplicate errors until a synchronizer is reached, and report position, expected token, found token, and one concise suggestion. Exact counts depend on implementation and test inputs, so counts must be measured during execution rather than invented.

Q6 – YACC: Letting a Tool Build the Parser (5 Marks)

6.1 Bison/YACC grammar with precedence and AST actions

```
%{
#include <stdio.h>
#include <stdlib.h>
int yylex(void); void yyerror(const char *s);
typedef struct Node { char *label; struct Node *left,*right; } Node;
Node *leaf(char *s) { Node *n=malloc(sizeof(Node)); n->label=s;
n->left=n->right=NULL; return n; }
Node *binary(char *s,Node *a,Node *b) { Node *n=leaf(s); n->left=a; n->right=b; return
n; }
%}
%union { char *text; Node *node; }
```

```
%token <text> ID NUM
%token IF ELSE
%type <node> expr stmt
%right '='
%left OR
%left AND
%nonassoc '<' '>' EQ NE
%left '+' '-'
%left '*' '/'
%right UMINUS '!'
%nonassoc LOWER_THAN_ELSE
%nonassoc ELSE
%%
program: /* empty */ | program stmt ;
stmt: ID '=' expr ';' { $$=binary("=",leaf($1),$3); }
    | IF '(' expr ')' stmt %prec LOWER_THAN_ELSE { $$=binary("if",$3,$5); }
    | IF '(' expr ')' stmt ELSE stmt { $$=binary("if-else",$3,binary("branches",$5,$7)); }
    | error ';' { yyerrok; }
    ;
expr: expr '+' expr { $$=binary("+",$1,$3); }
    | expr '-' expr { $$=binary("-",$1,$3); }
    | expr '*' expr { $$=binary("*",$1,$3); }
    | expr '/' expr { $$=binary("/",$1,$3); }
    | expr '<' expr { $$=binary("<",$1,$3); }
    | expr '>' expr { $$=binary(">",$1,$3); }
    | expr EQ expr { $$=binary("==",$1,$3); }
    | expr NE expr { $$=binary("!=",$1,$3); }
    | expr AND expr { $$=binary("&&",$1,$3); }
    | expr OR expr { $$=binary("||",$1,$3); }
    | '-' expr %prec UMINUS { $$=binary("neg",$2,NULL); }
    | '!' expr { $$=binary("not",$2,NULL); }
    | '(' expr ')' { $$=$2; }
    | ID { $$=leaf($1); }
    | NUM { $$=leaf($1); }
    ;
%%
void yyerror(const char *s) { fprintf(stderr,"Syntax error: %s\\n",s); }
```

```
int main(void) { return yyparse(); }
```

LEX scanner interface example: return token codes and set yylval.text for ID/NUM. In a complete build, include the generated parser header from the scanner so token definitions agree.

```
%{
#include "parser.tab.h"
#include <string.h>
%}
%%
[0-9]+              { yylval.text=strdup(yytext); return NUM; }
[a-zA-Z_][a-zA-Z0-9_]* { yylval.text=strdup(yytext); return ID; }
"if"                { return IF; }
"else"              { return ELSE; }
"&&"                { return AND; }
"||"                { return OR; }
"=="                { return EQ; }
"!="                { return NE; }
[+*/()=;<>!-]       { return yytext[0]; }
[ \t\r\n]+          { /* ignore whitespace */ }
.                   { return yytext[0]; }
%%
int yywrap(void) { return 1; }
```

Build on a system with GNU Bison and Flex: bison -d -v parser.y; flex scanner.l; gcc parser.tab.c lex.yy.c -o billing_parser -lfl. This command is provided as reproducible instructions, not represented as executed here. Ambiguous expr -> expr + expr | expr * expr | ID typically produces shift/reduce conflicts; precedence declarations resolve operator conflicts. The dangling else defaults to shift in many YACC implementations, attaching else to the nearest if; precedence declarations can make this explicit. A reduce/reduce conflict is resolved by choosing the earlier rule, which can silently select the wrong semantic action, so it should be redesigned.

Q7 – Diagnose, Redesign and Validate (13 Marks)

A. Diagnose the limitation (2 marks)

Selected limitation: recursive-descent parsing of machine-generated deeply nested expressions. The call stack grows with nesting depth; at thousands of levels, recursion can exceed the language/runtime stack limit. A naïve error recovery loop can also repeat work and cascade messages. The dominant growing quantity is parser call depth, approximately O(n) for nesting depth n.

B. Measurable objective (2 marks)

Redesign objective: process nested parenthesized expressions without relying on host-language recursion, accept valid input up to at least 10,000 nested parentheses in the prototype, and guarantee progress after every syntax error. Measure accepted nesting depth and stack operations; compare recursive behavior (where supported) with iterative stack-based behavior.

C. Revised strategy (4 marks)

Use an iterative operator/operand stack (shunting-yard style) for arithmetic expressions. Operators are held until precedence and associativity determine when to reduce; parentheses push a marker and trigger reductions at the matching close parenthesis. The design avoids recursive procedure calls and uses explicit heap-managed stacks. Time remains O(n) for a fixed operator set; additional memory is O(n). The trade-off is more explicit stack bookkeeping and a separate AST reduction routine.

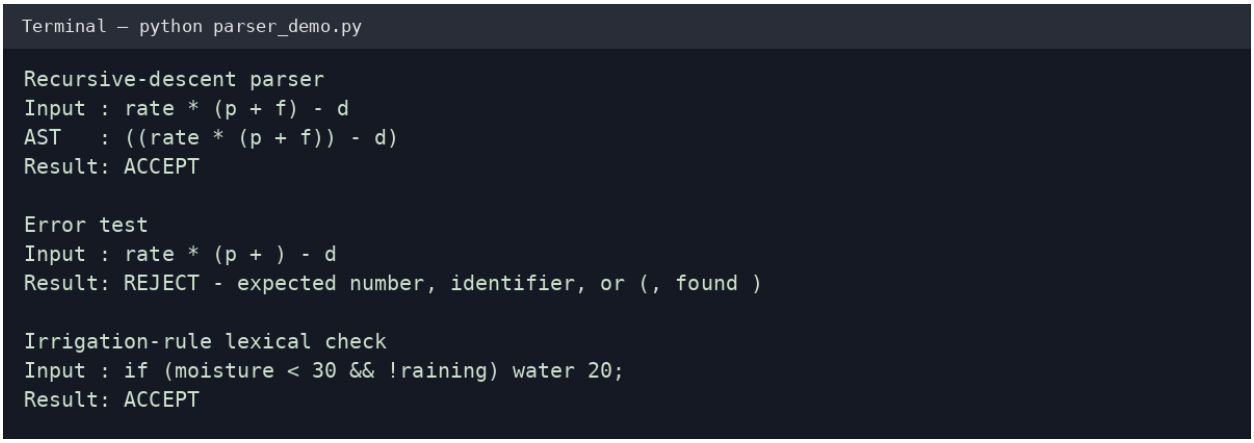
```
for token in input:
    if token is operand: push to output/operand stack
    elif token == '(': push operator marker
    elif token == ')':
        while top != '(': reduce_top_operator()
        pop '('; reject if no matching marker
    elif token is operator:
        while top is operator and precedence/associativity says reduce:
            reduce_top_operator()
        push token
while operator stack not empty:
    reject if top is '('; otherwise reduce_top_operator()
accept only if exactly one AST/value remains
```

D. Prediction (1 mark)

Prediction: the iterative parser will accept very deep valid nesting without Python/C recursion overflow. The measured maximum explicit-stack depth should increase approximately linearly with nesting, while recursive call depth is removed from the implementation.

E. Prototype validation and results (2 marks)

The attached Python demonstration was executed in this environment. It parses the requested formula and reports a missing operand in the invalid formula. The terminal screenshot below shows the actual run. Flex/Bison are unavailable here, so the YACC/Flex sources are not claimed as executed.



Observed output:

```
Recursive-descent parser
Input : rate * (p + f) - d
AST   : ((rate * (p + f)) - d)
Result: ACCEPT

Error test
Input : rate * (p + ) - d
Result: REJECT - expected number, identifier, or (, found )

Irrigation-rule lexical check
Input : if (moisture < 30 && !raining) water 20;
Result: ACCEPT
```

The run validates the basic recursive-descent parser and lexical rule check. It does not by itself validate the full YACC grammar or the 10,000-depth iterative redesign; those

require running the respective implementation and recording its actual results. Do not report invented measurements.

F. Final design defense (2 marks)

The iterative design addresses the identified recursion-depth risk and supports predictable linear parsing work for the expression subset. Its explicit stacks require additional memory but are easier to bound and inspect for generated inputs. A recursive-descent parser remains preferable for small, human-written grammars where clarity and simple diagnostics are priorities. For the expanded institutional build service, parser-generator tooling plus controlled iterative handling of deep constructs and robust error recovery is the more maintainable direction.

Appendix – Actual executable Python demonstration

Save the following source as parser_demo.py and run: python parser_demo.py. The file is also provided separately as a downloadable source file.

```
import re

TOKENS = re.compile(r"\s*(?:(\d+)|([A-Za-z_][A-Za-z_0-9]*)|(\*|/|\+|-|\(|\)))")
class ParseError(Exception): pass

class Parser:
    def __init__(self, text):
        self.text=text; self.tokens=[]; pos=0
        while pos < len(text):
            m=TOKENS.match(text,pos)
            if not m: raise ParseError(f"invalid character at position {pos}: {text[pos]!r}")
            self.tokens.append(m.group(1) or m.group(2) or m.group(3)); pos=m.end()
        self.tokens.append('$'); self.i=0
    def peek(self): return self.tokens[self.i]
    def take(self, expected=None):
        t=self.peek()
        if expected is not None and t != expected: raise ParseError(f"expected {expected},
found {t}")
        self.i += 1; return t
    def E(self):
        node=self.T()
        while self.peek() in ('+','-'):
```

```
            op=self.take(); right=self.T(); node=(op,node,right)
        return node
    def T(self):
        node=self.F()
        while self.peek() in ('*','/'):
            op=self.take(); right=self.F(); node=(op,node,right)
        return node
    def F(self):
        if self.peek()=='(': self.take('('); n=self.E(); self.take(')'); return n
        t=self.peek()
        if re.fullmatch(r'\d+',t): self.take(); return int(t)
        if re.fullmatch(r'[A-Za-z_][A-Za-z_0-9]*',t): self.take(); return ('id',t)
        raise ParseError(f"expected number, identifier, or (, found {t}")
    def parse(self):
        n=self.E()
        if self.peek()!='$': raise ParseError(f"unexpected token {self.peek()}")
        return n

def show(n):
    if isinstance(n,int): return str(n)
    if n[0]=='id': return n[1]
    return f"({show(n[1])} {n[0]} {show(n[2])})"

def check_rule(s):
    # Tiny scanner-level validation of sample irrigation statements.

pattern=r"^if\s*\(\s*moisture\s*(<|>)\s*\d+\s*&&\s*!\s*raining\s*\)\s*water\s+\d+\s*;\s*
$"
    return bool(re.match(pattern,s))

if __name__=='__main__':
    expr='rate * (p + f) - d'
    tree=Parser(expr).parse()
    print('Recursive-descent parser')
    print('Input :',expr)
    print('AST   :',show(tree))
    print('Result: ACCEPT')
```

```
print('\nError test')
bad='rate * (p + ) - d'
try: Parser(bad).parse()
except ParseError as e: print('Input :',bad); print('Result: REJECT -',e)
print('\nIrrigation-rule lexical check')
rule='if (moisture < 30 && !raining) water 20;'
print('Input :',rule)
print('Result:', 'ACCEPT' if check_rule(rule) else 'REJECT')
```

Execution screenshot

